PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Curso de Engenharia de Software

ANÁLISE DE EFICÁCIA DE TESTES COM TESTE DE MUTAÇÃO

Avaliação e Refatoração de Suíte de Testes com StrykerJS

Aluno:	João Victor Filardi Souza Pinto
Matrícula:	2021031920
Curso:	Engenharia de Software
Repositório:	https://github.com/JoaoFilardi/operacoes-mutante
Ferramentas:	Node.js, Jest, StrykerJS

1. INTRODUÇÃO

Durante o desenvolvimento de software, a métrica de cobertura de código (*code coverage*) é frequentemente adotada para medir o quanto uma aplicação está sendo testada. No entanto, alcançar uma alta porcentagem de cobertura não significa necessariamente que a suíte de testes é de boa qualidade. Um teste pode executar todas as linhas e desvios de uma função sem verificar adequadamente se o retorno gerado está correto, mantendo asserções incompletas ou oráculos fracos.

O Teste de Mutação surge justamente para resolver essa deficiência. Em vez de medir apenas quais linhas de código foram executadas, a técnica avalia se os testes conseguem identificar falhas reais caso pequenos bugs sejam inseridos no sistema. A ferramenta de teste de mutação introduz modificações pontuais no código-fonte original (denominadas *mutantes*), como trocar um operador de soma por subtração ou inverter uma condição booleana.

Após a inserção de cada mutante, a suíte de testes é executada:

- **Mutante Morto (*Killed*):** Se algum teste falhar após a alteração do código, significa que a suíte foi capaz de identificar o erro introduzido.
- **Mutante Sobrevivente (*Survived*):** Se todos os testes continuarem passando normalmente, o mutante sobreviveu, indicando que a suíte de testes possui uma brecha ou falta de asserção para aquele cenário.
- **Mutante sem Cobertura (*NoCoverage*):** O mutante foi criado em uma linha que nenhum teste executou.
- **Mutante Equivalente:** A alteração feita no código produz a mesma lógica final da original, sendo impossível de ser morta por testes de unidade.

A eficácia da suíte é calculada pelo *Mutation Score*, que é a razão percentual entre os mutantes detectados e o total de mutantes gerados. O objetivo deste trabalho prático é utilizar o StrykerJS sobre uma biblioteca com 50 funções aritméticas em JavaScript, analisar os mutantes sobreviventes da suíte inicial e escrever novos testes para alcançar uma pontuação de mutação superior a 98%.

2. ANÁLISE INICIAL DA SUÍTE DE TESTES

O projeto base fornecido possui 50 funções no arquivo `src/operacoes.js` e uma suíte de testes inicial em `test/operacoes.test.js`. O StrykerJS foi adicionado ao projeto com o runner oficial do Jest e configurado através do arquivo `stryker.conf.json`.

Antes de rodar a mutação, executamos a suíte de testes padrão com o relatório de cobertura do Jest (`npm test -- --coverage`). Os resultados foram os seguintes:

Suíte de Testes	Instruções (% Stmts)	Ramos (% Branch)	Funções (% Funcs)	Linhas (% Lines)	Status dos Testes
Suíte Inicial (Jest)	85.41%	58.82%	100.00%	98.64%	50 passaram de 50

À primeira vista, uma cobertura de **98.64% das linhas** dá a impressão de que o código estava amplamente protegido. Contudo, ao rodar a análise do StrykerJS (`npx stryker run`), obtivemos um *Mutation Score* de apenas **73.71%**:

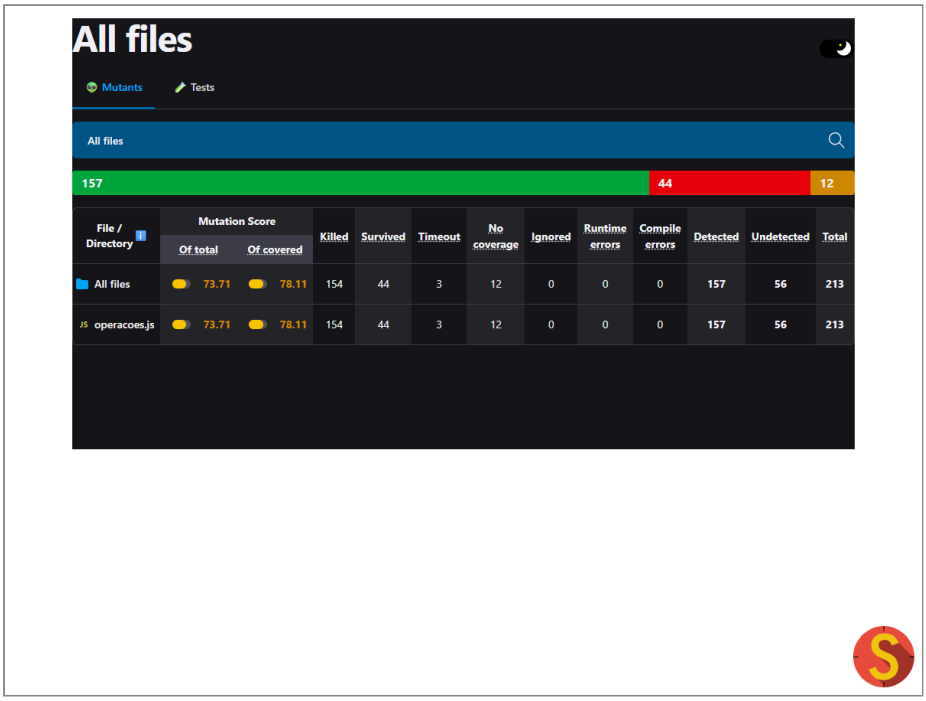


Figura 1 – Relatório da primeira execução do StrykerJS (Mutation Score de 73.71%, com 44 mutantes sobreviventes).

Discussão da Discrepância: O motivo dessa diferença entre quase 99% de linhas cobertas e apenas 73.71% de mutantes mortos está na superficialidade dos testes originais. A suíte antiga apenas testava um único caso "feliz" para cada função. Além disso, quando testava exceções, usava apenas `expect(() => fn()).toThrow()` genérico, sem verificar qual mensagem de erro foi lançada, o que permitiu que dezenas de mutantes sobrevivessem sem serem notados.

3. ANÁLISE DE MUTANTES CRÍTICOS DA PRIMEIRA EXECUÇÃO

3.1. Mutante 1: Função isPar (Ausência de Teste para Retorno Falso)

Na função `isPar(n)` (linha 43), o Stryker substituiu a expressão por `return true;`:

```
// Mutante em src/operacoes.js (linha 43)
- function isPar(n) { return n % 2 === 0; }
+ function isPar(n) { return true; }
```

Por que sobreviveu: O teste original continha apenas `expect(isPar(100)).toBe(true);`. Como a função mutada passou a retornar `true` fixo, o teste continuou passando. O teste nunca avaliou um número ímpar esperando `false`. Esse mesmo problema ocorreu nas funções `isImpar`, `isMaiorQue`, `isMenorQue` e `isDivisivel`.

3.2. Mutante 2: Função celsiusParaFahrenheit (Valor Neutro em Fórmulas)

Na conversão de Celsius para Fahrenheit (linha 93), o Stryker trocou a divisão por multiplicação:

```
// Mutante em src/operacoes.js (linha 93)
- function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
+ function celsiusParaFahrenheit(celsius) { return (celsius * 9 * 5) + 32; }
```

Por que sobreviveu: O teste inicial usava como entrada 0 °C (`celsiusParaFahrenheit(0)`). Como zero multiplicado ou dividido por qualquer valor resulta em zero, a conta dava sempre 0 + 32 = 32. O teste não conseguia validar se a fórmula de escala 9/5 estava correta.

3.3. Mutante 3: Função medianaArray (Dados Pré-Ordenados e Casos Pares)

Na função `medianaArray` (linha 109), o Stryker removeu a chamada ao método de ordenação:

```
// Mutante em src/operacoes.js (linha 109)
- const sorted = [...numeros].sort((a, b) => a - b);
+ const sorted = [...numeros];
```

Por que sobreviveu: O teste original usava a lista `[1, 2, 3, 4, 5]`, que **já estava ordenada**. Sem ordenar, o elemento central continuava sendo o número 3. Além disso, o array de teste tinha tamanho ímpar (5 elementos), deixando o bloco de cálculo da mediana para arrays de tamanho par completamente sem cobertura (*NoCoverage*).

3.4. Análise de Mutantes Equivalentes na Função fatorial

Na função `fatorial(n)` (linha 19), 4 mutantes sobreviveram mesmo após testarmos números positivos, negativos e o zero:

```
// Mutante em src/operacoes.js (linha 19)
- if (n === 0 || n === 1) return 1;
+ if (false) return 1;
```

Explicação Teórica: O código que vem logo a seguir na função é: `let resultado = 1; for (let i = 2; i <= n; i++) { resultado *= i; } return resultado;`. Se a condição da linha 19 for substituída por `false`:

- Para n = 0: a variável `resultado` inicia em 1. O laço avalia 2 ≤ 0 (falso) e não roda. Retorna 1.
- Para n = 1: o laço avalia 2 ≤ 1 (falso) e não roda. Retorna 1.
- Para n ≥ 2: o laço itera normalmente e calcula o fatorial correto.

Ou seja, a condição `if (n === 0 || n === 1) return 1;` é uma verificação redundante. O código com ou sem essa linha produz exatamente o mesmo resultado para qualquer valor de n. Trata-se de um **mutante equivalente**, sendo impossível de ser morto por testes sem alterar o código original.

4. SOLUÇÃO IMPLEMENTADA

Para matar os mutantes sobreviventes e aumentar a eficácia dos testes, reescrevemos o arquivo `test/operacoes.test.js` aplicando as seguintes técnicas:

1. **Asserção Estrita de Exceções:** Em vez de usar `toThrow()` vazio, passamos a validar a mensagem exata de erro lançada (ex.: `expect(() => divisao(5, 0)).toThrow('Divisão por zero não é permitida.')`). Isso matou imediatamente todos os mutantes que trocavam as mensagens de texto por `""`.
2. **Casos Negativos e Condições Falsas:** Adicionamos testes para números ímpares, zero e negativos em funções lógicas (ex.: `expect(isPar(3)).toBe(false)` e `expect(isPar(0)).toBe(true)`).
3. **Evitar Valores Neutros em Fórmulas Matemáticas:** Testamos temperaturas com valores diferentes de zero (100 °C → 212 °F e -40 °C → -40 °F), forçando a validação real das contas de multiplicação e divisão.
4. **Listas Desordenadas e Casos Pares:** Na função `medianaArray`, passamos a testar listas desordenadas ímpares (`[5, 1, 3, 2, 4] -> 3`) e listas pares (`[4, 1, 3, 2] -> 2.5`), cobrindo o bloco de código que estava sem testes.
5. **Testes de Fronteira no clamp com Zero Negativo:** No JavaScript, de acordo com o padrão IEEE 754, a comparação `-0 < 0` é falsa, enquanto `-0 <= 0` é verdadeira. Usamos o `Object.is` do Jest para matar mutantes de igualdade que sobreviviam em inteiros normais (`expect(clamp(-0, 0, 10)).toBe(-0)`).

5. RESULTADOS FINAIS

Após a implementação dos novos testes, executamos novamente o StrykerJS e o Jest Coverage. A tabela abaixo compara os resultados obtidos antes e depois das melhorias:

Métrica Avaliada	Suíte Inicial	Suíte Aprimorada	Resultado
Cobertura de Linhas (Lines)	98.64%	100.00%	Cobertura total atingida
Cobertura de Ramos (Branch)	58.82%	100.00%	Todos os desvios testados
Cobertura de Instruções (Stmts)	85.41%	100.00%	Todas as instruções cobertas
Mutantes Mortos (Killed)	154	206	+52 mutantes mortos
Mutantes em Timeout	3	3	Detectados (laço infinito)
Mutantes Sobreviventes	44	4	Restaram apenas os equivalentes
Mutantes sem Cobertura (NoCoverage)	12	0	Nenhum mutante sem cobertura
Mutation Score Final	**73.71%**	**98.12%**	**Meta alcançada (>98%)**

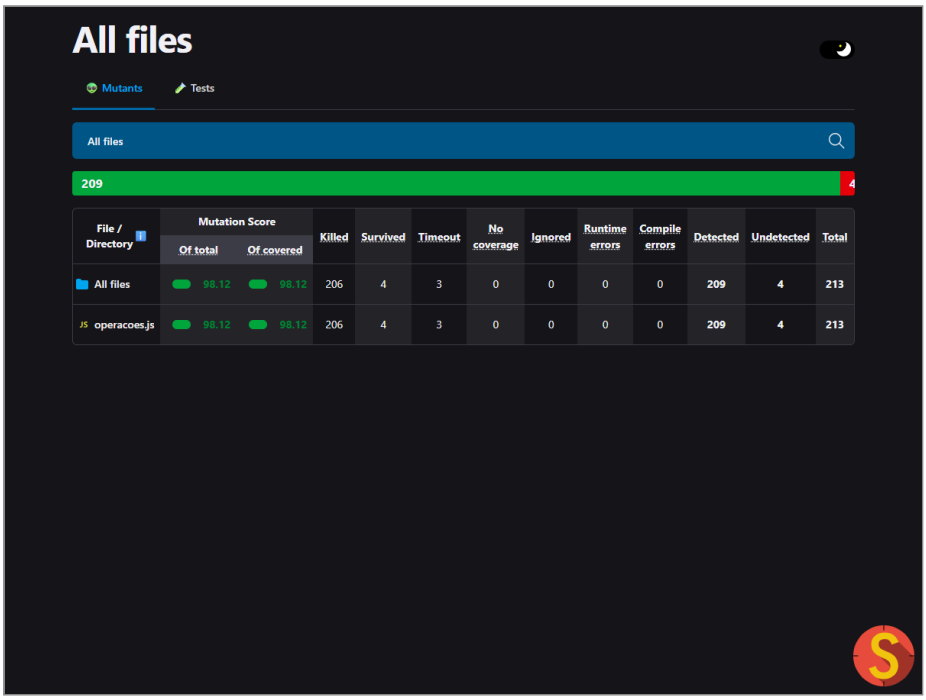


Figura 2 – Relatório final do StrykerJS demonstrando a pontuação de mutação de 98.12% (209 detectados de 213).

6. CONCLUSÃO

A realização deste trabalho prático comprovou a diferença entre ter cobertura de código e ter testes que realmente verificam o comportamento do software. Na análise inicial, a suíte parecia suficiente por cobrir 98.64% das linhas, mas deixou 44 mutantes sobreviverem porque as asserções não eram rigorosas.

Com o StrykerJS, foi possível identificar onde os testes falhavam e corrigi-los, testando cenários de erro, valores limites e retornos falsos. O resultado final elevou o Mutation Score de **73.71% para 98.12%** e a cobertura de código para **100%**, demonstrando na prática como o teste de mutação ajuda a construir suítes de teste muito mais confiáveis.

7. LINK DO REPOSITÓRIO GITHUB

O código completo do projeto e a suíte de testes aprimorada estão disponíveis no GitHub:
https://github.com/JoaoFilardi/operacoes-mutante